Arrays in C++

Introduction to Arrays **1**
Range-Based for Loop with Array 7
Range-Based for Loop — Without Reference **8**
Range-Based for Loop — With Reference **9**
Range-Based for Loop — With const Reference **11**
Passing Array to Function **17**
Multi-Dimensional Arrays (Brief Introduction) **19**
Advantages of Arrays **21**
Disadvantages of Arrays **21**
Common Operations on Arrays **21**
Sum of Elements 21
Find Maximum Element 22
Find Minimum Element 23
Reverse an Array 24

Introduction to Arrays

An **array** is a collection of elements of the **same data type** stored in **contiguous memory locations**.

Instead of creating multiple variables for similar data, an array stores all the values using a single variable name.

Without Array
```
int marks1 = 80;
int marks2 = 75;
int marks3 = 90;
int marks4 = 85;
int marks5 = 95;
```

This becomes difficult to manage.

With Array
```
int marks[5] = {80, 75, 90, 85, 95};
```

Here,

- marks is the array name
- 5 is the size
- Each element has an index.

Characteristics of Array

- Stores multiple values of same data type.
- Stored in contiguous memory locations.
- Size is fixed after declaration.
- Index starts from 0.
- Random access is possible.
- Faster access because memory is continuous.

Memory Representation

Suppose

int arr[5] = {10,20,30,40,50};

Memory

Address	value	array name
1000	10	arr[0]
1004	20	arr[1]
1008	30	arr[2]
1012	40	arr[3]
1016	50	arr[4]

Each integer occupies **4 bytes**.

Address calculation

Address = Base Address + (Index × Size of Data Type)

Example

Base Address = 1000
Size of int = 4

arr[3]

Address: 1000 + (3 × 4) = 1012

Syntax of Array Declaration

datatype array_name[size];

Example

int marks[5];

float salary[10];

char name[20];

double price[50];

Different Ways of Declaring and Initializing Arrays

Method 1: Declaration Only

int arr[5];

Memory is allocated.

Values are garbage values (local array).

Index	**value**
arr[0]	Garbage
arr[1]	Garbage
arr[2]	Garbage
arr[3]	Garbage
arr[4]	Garbage

Method 2: Declaration with Initialization

int arr[5] = {10,20,30,40,50};

Output

10 20 30 40 50

Method 3: Size Automatically Determined

Compiler counts the number of elements.

int arr[] = {5,10,15,20};

Size becomes 4.

Method 4: Partial Initialization

int arr[5] = {10,20};

Remaining elements become zero.

Index	value
arr[0]	10
arr[1]	20
arr[2]	0
arr[3]	0
arr[4]	0

Method 5: Initialize All Elements to Zero

int arr[5] = {};

or

int arr[5] = {0};

Output

0
0
0
0
0

Method 6: User Input

```
#include<iostream>
using namespace std;
```

```
int main()
{
   int arr[5];

   for(int i=0;i<5;i++)
   {
       cin>>arr[i];
   }

   for(int i=0;i<5;i++)
   {
       cout<<arr[i]<<" ";
   }
}
```

Input

5
10
15
20
25

Output

5 10 15 20 25

Method 7: Using Loop for Initialization

```
int arr[5];

for(int i=0;i<5;i++)
{
   arr[i]=i*10;
}
```

Output

0 10 20 30 40

Accessing Array Elements

Syntax

array_name[index]

Example

int arr[5]={10,20,30,40,50};

cout<<arr[0]<<endl;

cout<<arr[2]<<endl;

cout<<arr[4];

Output :

10
30
50

Modifying Array Elements

int arr[5]={10,20,30,40,50};

arr[2]=100;

Output

10 20 100 40 50

Traversing an Array

Using for Loop

```
#include<iostream>
using namespace std;

int main()
{
    // Declare and initialize an integer array with 5 elements
```

```
    int arr[] = {10, 20, 30, 40, 50};

    // Loop through each element of the array
    // i starts from 0 and continues until i becomes 4
    for(int i = 0; i < 5; i++)
    {
        // Print the current array element
        cout << arr[i] << " ";
    }

    return 0;   // Indicate successful program execution
}
```

Output:
10 20 30 40 50

Range-Based `for` Loop with Array

C++ provides a **range-based `for` loop** to easily access every element of an array.

The basic syntax is:

```
for(data_type variable : array)
{
    // statements
}
```

For example:

```
#include <iostream>
using namespace std;

int main()
{
    // Declare and initialize an integer array
    int arr[] = {10, 20, 30, 40, 50};
```

```
    // Range-based for loop
    // 'x' takes the value of each element of the array one by one
    for(int x : arr)
    {
        // Print the current element
        cout << x << " ";
    }

    // Return 0 indicates successful program execution
    return 0;
}
```

Here, x receives the value of each array element one by one

Range-Based `for` Loop — Without Reference

```
#include <iostream>
using namespace std;

int main()
{
    // Declare and initialize an integer array
    int arr[] = {10, 20, 30};

    // Range-based for loop
    // x is a separate variable that receives a COPY
    // of each array element
    for(int x : arr)
    {
        // Add 10 to x
        x = x + 10;

        // This changes only the local variable x,
        // not the original array element
    }

    // Display the original array
    for(int x : arr)
    {
```

```
        cout << x << " ";
    }

    return 0;
}
```

Output :
10 20 30

Why does the array not change?

Consider: for(int x : arr)

Suppose the array is: arr = {10, 20, 30}

During the loop:

First iteration:
x = 10 // copy of arr[0]

Second iteration:
x = 20 // copy of arr[1]

Third iteration:
x = 30 // copy of arr[2]

When we execute:

x = x + 10;

the value of x changes, but the original array does not. **Without &, the loop variable is a copy of the array element.**

Range-Based `for` Loop — With Reference

If we want to **modify the original array elements**, use a reference &.

```
#include <iostream>
using namespace std;

int main()
```

```
{
    // Declare and initialize an integer array
    int arr[] = {10, 20, 30};

    // Range-based for loop with reference
    // x becomes a reference to the actual array element
    for(int &x : arr)
    {
        // Add 10 to the original array element
        x = x + 10;
    }

    // Display the modified array
    for(int x : arr)
    {
        cout << x << " ";
    }

    return 0;
}
```

Output
20 30 40

Why does the array change?

Here:

for(int &x : arr)

x does **not** create a copy.

Instead, x refers directly to the original element.

arr[0] = 10
  ↑
  |
  x

Therefore:

x = x + 10;

is effectively modifying: arr[0] = arr[0] + 10;

The iterations are:

arr[0] = 10 → x refers to arr[0] → arr[0] becomes 20

arr[1] = 20 → x refers to arr[1] → arr[1] becomes 30

arr[2] = 30 → x refers to arr[2] → arr[2] becomes 40

Final array:

{20, 30, 40}

Important

int x means **copy the value**.
int &x means **refer to the original value**.

Range-Based for Loop — With const Reference

Sometimes we don't want to modify the array, but we also don't want to create a copy of every element.

In that case, use:

const int &x

Example:

```
#include <iostream>
using namespace std;

int main()
{
    // Declare and initialize an integer array
    int arr[] = {10, 20, 30};

    // Range-based for loop with const reference
    // x refers to the original element,
    // but x cannot be used to modify that element
    for(const int &x : arr)
    {
        // Display the current element
```

```
        cout << x << " ";
    }

    return 0;
}
```

Output

10 20 30

Because `x` is `const`, this is **not allowed**:

```
for(const int &x : arr)
{
   x = x + 10;   // ERROR
}
```

The compiler prevents modification through `x`.

Why Use `const int &`?

Consider a large object or large data type.

With:

```
for(int x : arr)
```

each element is copied into `x`.

With:

```
for(const int &x : arr)
```

no copy of the element is made. `x` refers directly to the existing element, but cannot modify it.

So:

```
const int &x
```

provides two advantages:

1. **Avoids unnecessary copying**
2. **Prevents accidental modification**

For simple `int` arrays, the performance difference is usually insignificant, but the technique becomes more useful with **large objects, structures, and classes**.

Comparison

Loop	Copy/Reference	Can modify original?	Can modify `x`?
`for(int x : arr)`	Copy	No	Yes
`for(int &x : arr)`	Reference	Yes	Yes
`for(const int &x : arr)`	Const reference	No	No

Easy way to remember
int x

Copy

int &x

Reference → Can modify original

const int &x

Reference → Read only

Complete Example Showing All Three

```
#include <iostream>
using namespace std;

int main()
{
    int arr[] = {10, 20, 30};

    // ------------------------------------------------
    // 1. Without reference
    // ------------------------------------------------

    // x is a COPY of each array element.
    // Changes made to x do not affect the array.
```

```
    for(int x : arr)
    {
        x = x + 10;
    }

    cout << "After without reference: ";

    for(int x : arr)
    {
        cout << x << " ";
    }

    cout << endl;


    // ------------------------------------------------
    // 2. With reference
    // ------------------------------------------------

    // x is a REFERENCE to each original array element.
    // Therefore, changes made to x affect the array.
    for(int &x : arr)
    {
        x = x + 10;
    }

    cout << "After with reference: ";

    for(int x : arr)
    {
        cout << x << " ";
    }

    cout << endl;


    // ------------------------------------------------
    // 3. With const reference
    // ------------------------------------------------

    // x is a reference to the original element,
    // but const prevents modification.
```

```
    for(const int &x : arr)
    {
        cout << x << " ";
    }

    cout << endl;

    return 0;
}
```

Output

After without reference: 10 20 30
After with reference: 20 30 40

Finding Size of Array

int arr[]={10,20,30,40,50};
cout<<sizeof(arr);

Output : 20

Because 5 × 4 =20 bytes

Number of elements :

int size=sizeof(arr)/sizeof(arr[0]);

cout<<size;

Output : 5

Array Input and Output Example

```
#include<iostream>
using namespace std;

int main()
{
    // Declare an integer array that can store 5 elements
    int arr[5];
```

```
    // Display a message asking the user to enter array elements
    cout << "Enter elements: ";

    // Loop to read 5 elements from the user
    for(int i = 0; i < 5; i++)
    {
        // Store the user-entered value at index i
        cin >> arr[i];
    }

    // Display a message before printing the array elements
    cout << "Elements are: ";

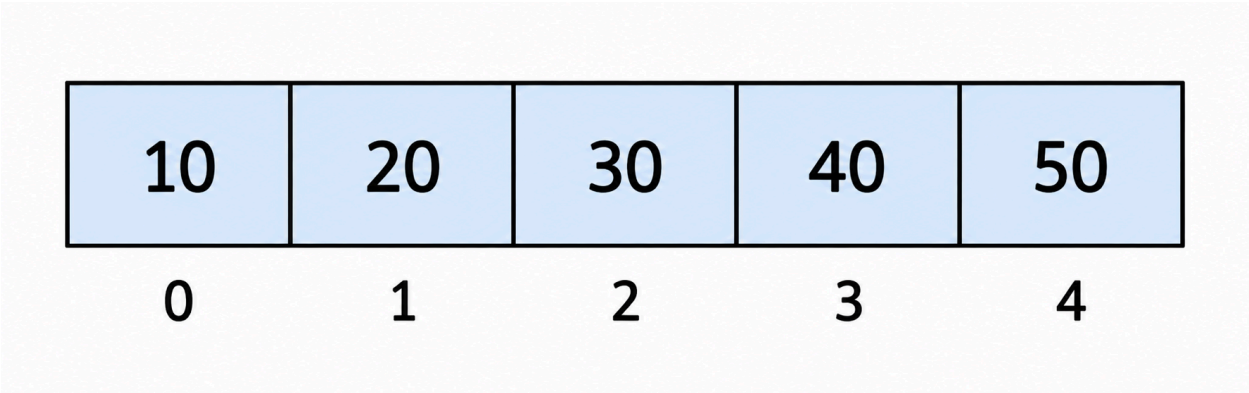
    // Loop through the array to display all 5 elements
    for(int i = 0; i < 5; i++)
    {
        // Print the current array element
        cout << arr[i] << " ";
    }

    // Return 0 to indicate successful program execution
    return 0;
}
```

Array Index

Array Index:

0
1
2
3
4



Passing Array to Function

```
#include<iostream>
using namespace std;
// Function to display all elements of the array
// arr  -> stores the address of the first element of the array
// size -> stores the number of elements in the array
void display(int arr[], int size)
{
    // Loop through the array from index 0 to size-1
    for(int i = 0; i < size; i++)
    {
        // Display the current array element
        cout << arr[i] << " ";
    }
}

int main()
{
    // Declare and initialize an integer array
    int arr[] = {10, 20, 30, 40, 50};

    // Call the display() function
    // Pass the array and its size as arguments
    display(arr, 5);

    return 0;
}
```

Output :
10 20 30 40 50

Modifying Array Inside Function

```cpp
#include<iostream>
using namespace std;

// Function to add 5 to every element of the array
// arr  -> receives the array
// size -> stores the number of elements in the array
void update(int arr[], int size)
{
    // Loop through each element of the array
    for(int i = 0; i < size; i++)
    {
        // Add 5 to the current array element
        arr[i] += 5;
    }
}

int main()
{
    // Declare and initialize an integer array
    int arr[] = {10, 20, 30};

    // Call the update() function
    // Pass the array and its size
    update(arr, 3);

    // Range-based for loop to display the updated array
    for(int x : arr)
    {
        // Print each element
        cout << x << " ";
    }

    return 0;
}
```

Output
15 25 35

Reason: When an array is passed to a function, it decays to a pointer to its first element, so changes made inside the function affect the original array.

Multi-Dimensional Arrays (Brief Introduction)

2D Array Declaration
int matrix[2][3];

Initialization

```
int matrix[2][3] =
{
  {1,2,3},
  {4,5,6}
};
```

Memory

1 2 3
4 5 6

Access

cout<<matrix[1][2];

Output : 6

Program

```
#include <iostream>
using namespace std;

int main()
```

```
{
    // Declare a 2D array with 3 rows and 3 columns
    int arr[3][3];

    // Take elements from the user
    cout << "Enter 9 elements:" << endl;

    for(int i = 0; i < 3; i++)
    {
        // Loop through columns
        for(int j = 0; j < 3; j++)
        {
            cin >> arr[i][j];
        }
    }

    // Display the 2D array
    cout << "\n2D Array:" << endl;

    for(int i = 0; i < 3; i++)
    {
        // Loop through columns
        for(int j = 0; j < 3; j++)
        {
            cout << arr[i][j] << " ";
        }

        // Move to the next row
        cout << endl;
    }
```

```
    return 0;
}
```

Advantages of Arrays

- Easy to store multiple values.
- Fast random access using index.
- Less code compared to multiple variables.
- Contiguous memory improves cache performance.
- Convenient for sorting and searching algorithms.

Disadvantages of Arrays

- Fixed size; it cannot grow or shrink after creation.
- Stores only one data type.
- Insertion and deletion in the middle are costly because elements need to be shifted.
- May waste memory if allocated size is larger than needed.

Common Operations on Arrays

Sum of Elements

```
#include <iostream>
using namespace std;

int main()
{
    // Array declaration and initialization
    int arr[] = {10, 20, 30, 40, 50};

    // Variable to store sum
```

```
    int sum = 0;

    // Range-based for loop
    for (int x : arr)
    {
        sum += x;   // Add each element to sum
    }

    // Display the result
    cout << "Sum = " << sum;

    return 0;
}
```

Output

150

Find Maximum Element

```
#include <iostream>
using namespace std;

int main()
{
    // Array declaration and initialization
    int arr[] = {10, 55, 20, 40, 30};

    // Assume first element is the largest
    int max = arr[0];

    // Traverse the array
    for (int x : arr)
    {
        // Update max if a larger element is found
        if (x > max)
        {
            max = x;
        }
    }
```

```
    // Display the largest element
    cout << "Maximum = " << max;

    return 0;
}
```

Output

55

Find Minimum Element

```
#include <iostream>
using namespace std;

int main()
{
    // Array declaration and initialization
    int arr[] = {10, 55, 20, 40, 30};

    // Assume the first element is the smallest
    int min = arr[0];

    // Traverse the array
    for (int x : arr)
    {
        // Update min if a smaller element is found
        if (x < min)
        {
            min = x;
        }
    }

    // Display the smallest element
    cout << "Minimum = " << min;

    return 0;
}
```

Output

10

Reverse an Array

```
#include<iostream>
using namespace std;

int main()
{
   int arr[]={10,20,30,40,50};
   int size=sizeof(arr)/sizeof(arr[0]);

   for(int i=size-1;i>=0;i--)
   {
       cout<<arr[i]<<" ";
   }
}
```

Output

50 40 30 20 10

Sorting of Array

```
#include <iostream>
using namespace std;

// Function to perform Bubble Sort
void bubbleSort(int arr[], int size)
{
    // Repeat the passes for all elements
    for(int i = 0; i < size - 1; i++)
    {
        // Compare adjacent elements
        for(int j = 0; j < size - i - 1; j++)
        {
            // If the current element is greater than
            // the next element, swap them
            if(arr[j] > arr[j + 1])
```

```
            {
                int temp = arr[j];
                arr[j] = arr[j + 1];
                arr[j + 1] = temp;
            }
        }
    }
}

// Function to perform Selection Sort
void selectionSort(int arr[], int size)
{
    // Move through each position of the array
    for(int i = 0; i < size - 1; i++)
    {
        // Assume the current position has the
        // smallest element
        int minIndex = i;

        // Search for the smallest element
        // in the remaining unsorted array
        for(int j = i + 1; j < size; j++)
        {
            if(arr[j] < arr[minIndex])
            {
                // Store the index of the smallest element
                minIndex = j;
            }
        }

        // Swap the smallest element with
        // the current element
        int temp = arr[i];
        arr[i] = arr[minIndex];
        arr[minIndex] = temp;
    }
}

// Function to display the array
void display(int arr[], int size)
{
    for(int i = 0; i < size; i++)
```

```
    {
        cout << arr[i] << " ";
    }

    cout << endl;
}

int main()
{
    int arr1[] = {64, 25, 12, 22, 11};
    int arr2[] = {64, 25, 12, 22, 11};

    int size = 5;

    // Display original array
    cout << "Original Array: ";
    display(arr1, size);

    // Perform Bubble Sort
    bubbleSort(arr1, size);

    cout << "After Bubble Sort: ";
    display(arr1, size);

    // Perform Selection Sort
    selectionSort(arr2, size);

    cout << "After Selection Sort: ";
    display(arr2, size);

    return 0;
}
```

Binary Search Operation

```
#include <iostream>
using namespace std;

// Function to perform Binary Search
int binarySearch(int arr[], int size, int key)
{
```

```
    int low = 0;
    int high = size - 1;

    // Continue searching while the range is valid
    while(low <= high)
    {
        // Find the middle index
        int mid = (low + high) / 2;

        // If middle element is the key, return its index
        if(arr[mid] == key)
        {
            return mid;
        }

        // If key is greater than middle element,
        // search in the right half
        if(key > arr[mid])
        {
            low = mid + 1;
        }
        // If key is smaller than middle element,
        // search in the left half
        else
        {
            high = mid - 1;
        }
    }

    // Return -1 if the element is not found
    return -1;
}

int main()
{
    // Binary Search requires a sorted array
    int arr[] = {10, 20, 30, 40, 50, 60, 70};

    int size = 7;
    int key;

    // Take the element to search from the user
```

```
    cout << "Enter element to search: ";
    cin >> key;

    // Call Binary Search function
    int result = binarySearch(arr, size, key);

    // Check whether the element was found
    if(result != -1)
    {
        cout << "Element found at index " << result;
    }
    else
    {
        cout << "Element not found";
    }

    return 0;
}
```

Questions

1. What is an array in C++?
2. Why does array indexing start from 0?
3. How are arrays stored in memory?
4. What is the difference between an array and a pointer?
5. How do you find the size of an array?
6. What is the difference between `int arr[5]` and `int arr[] = {1,2,3}`?
7. What happens when an array is passed to a function?
8. What is the difference between a traditional `for` loop and a range-based `for` loop?
9. Why use `int&` in a range-based `for` loop?
10. What are the limitations of arrays?